Community Detection Algorithms Comparison and Analysis

By Vincent Pestilli, Wesley Ni, and Marcus VanWerry
Github:
https://github.com/DoA26-VWLM/DoA26_Community_Detection_Algorithms_Comparisons

Abstract

This project implements and compares four approaches to community detection: brute force permutation, Label Propagation (LPA), Spectral Clustering, and the Emperor Penguin Optimization Algorithm (EPO). Each algorithm was evaluated against custom benchmark data and a series of randomly generated graphs of increasing size and density. Each algorithm was tested to determine its accuracy and runtime compared to the others. AI tooling was used throughout in conjunction with technical documentation to assist in code development, dataset generation, and experimentation. The results highlight the fundamental trade off between solution quality and computational cost, demonstrating the limitations of brute force at scale and the practical benefits of heuristic and propagation based methods for larger graphs.

Community Detection: Overview

Community detection is the process of identifying densely connected groups of nodes within a graph, referred to as communities. Our goal is to find sections of a network where internal connections are dense relative to external ones, and label them accordingly. This problem is generally considered NP-hard, meaning no known algorithm can guarantee an optimal solution in polynomial time for all cases. Community detection is also an inverse problem. Inverse problems by nature may not have a single exact solution, and even mathematically rigorous methods like brute force can produce varying results depending on how a community is formally defined. Some inconsistency across methods and runs is therefore expected. This project explores four algorithmic approaches to the problem. These methods vary significantly in complexity, accuracy, and scalability, and each represents a different philosophy for navigating the trade off between computational cost and solution quality. To test them, we use custom datasets and randomly generated graphs that scale in size and density, allowing us to observe how each algorithm degrades or holds up as the input size grows. The tests we ran helped us to benchmark both the empirical runtime and average solution quality of our algorithms.

Experimentation Goals and Questions

1. **Which algorithm scales best as the graph increases in size(up to n nodes)?**

   As n increases to infinity, Label Propagation runs fastest with approximately $\Theta(n)$ runtime. EPO also has near linear runtime, although, unlike LPA, it doesn't have an early ending case. This combined with more moving parts in EPO leads to LPA typically running faster than EPO.

2. **What is the trade off between solution quality and execution time?**

   There is a direct trade off between solution quality and execution time in community detection algorithms. In general, improving solution quality requires more computation, as the algorithm explores more possible partitions or performs additional refinement steps, increasing runtime. However, faster algorithms often achieve speed by using heuristics or local decisions, which can reduce accuracy. For example, the Label Propagation Algorithm is computationally efficient but may produce lower quality or less stable community structures due to its reliance on local majority updates and randomness rather than global optimization.

3. **How does parameter tuning affect computational cost?**

   Our algorithms do not require any explicit parameter tuning (spectral clustering excluded.) Brute force evaluates all possible partitions, so its computational cost is fixed based on input size. LPA and EPO do not rely on any manual parameters, but their behaviors are still greatly influenced by internal choices we made.

   In LPA, randomness plays an important role in tie breaks when a node has an equal number of each neighbor. While there were other more deterministic options, leaving it up to chance prevents bias. Other methods can affect convergence speed and accuracy, but overall computational cost remains similar.

   In EPO, some parameters we tuned were population size, and temperature cooling rate. These directly influence both runtime and accuracy. Larger populations or slower cooling increase computation cost and runtime, but may lead to more accurate solutions. This however does have diminishing returns, and we settled with a happy medium, balancing a reasonable runtime, with an acceptable accuracy.

Spectral clustering is the only algorithm we use with a clear external parameter, k, the number of communities. We named it K, as it is used directly by the K means clustering. This parameter directly scales the amount of computational work as more clusters require additional iterations and distance calculations.

Experimentation Details

We will be running our algorithms on increasingly large randomly generated datasets to gauge how well these algorithms perform at scale. We will time their actual runtime using the Python chrono library. We will also be using the modularity formula to compare solution qualities. The modularity formula mathematically gives us a score for the accuracy of a community.

The computer we will use to test these results has the following specs:

- CPU: 13th gen Intel i7-13700hx
- CPU Base Clock Speed: 2.10 Ghz
- Ram: 32 Gigabyte, 4800 MT/s
- GPU: Rtx 4060 laptop gpu

Our programming language of choice for this experiment was Python. Python has a lot of libraries with various functionalities. This means that a lot of heavy lifting and work for popular algorithms has already been done for us. We utilized libraries such as “networkx”, “numpy”, and “sklearn” to assist us in the development of our algorithms. Furthermore, we used ChatGPT to help us program, learn our algorithms, etc. This helped us overcome the time constraints and learning curve put upon us by our algorithms, especially our nature-inspired algorithm of EPO.

For our datasets, we used one real-life graph, which is the Zachary’s Karate Club dataset. It shows the split in a college karate club and is supposed to have 2 communities. For the rest of our datasets, we decided to use graphs randomly generated by a Python script that we programmed. Our original plan was to use AI to generate our datasets. However, ChatGPT would not generate large datasets. This led to us writing a script to allow us to create customizable datasets. We ended up using GML (Graph Modelling Language) for our datasets, which could be easily imported into a graph by “networkx” in Python. Our input size for all of our algorithms in n, which is the number of nodes in our graphs.

Algorithm Specifications:

- Brute-Force:
    - Input Size: n
    - Parameters: None
    - Metrics to Measure: Runtime
    - Number of trials: n=10 graph
    - Runtime: $\Theta(n!)$
- Label Propagation:
    - Input Size: n
    - Parameters: None (automatically gives 1 unique label per node)
    - Metrics to Measure: Runtime and solution quality
    - Number of Trials: Several due to randomness and varying solution quality
    - Runtime: $\Theta(n)$
- Spectral Clustering
    - Input Size: n
    - Parameters: Number of communities (k)
    - Metrics to Measure: Runtime and solution quality
    - Number of Trials: Several. There is a parameter, k, to tune. This also varies in runtime based on graph density.
    - Runtime: $\Theta(n^3)$
- Emperor Penguin Optimizer
    - Input Size: n
    - Parameters: None (Everything that would be a “parameter” is automatically scaled and determined)
    - Metrics to Measure: Runtime and solution quality
    - Number of Trials: Several. This algorithm involves randomness.
    - Runtime: $\Theta(n)$

Modularity

Modularity score is a metric that we used to evaluate the quality of a graph partition by measuring how strongly nodes within the same community are connected compared to what would be expected in a random graph with the same degree of distribution. Higher modularity indicates denser connections within communities and sparser connections between them, meaning a stronger community structure. It is commonly used as the objective function in community detection algorithms to compare and optimize different partitions. For our purposes, we use modularity in the Brute Force algorithm and EPO as a way of determining the quality of potential solutions.

Brute Force

The brute force algorithm detects communities by generating every permutation of partitions possible from the nodes, ranging from everything in one community, to everything in its own community. It then iterates through every permutation, and scores their modularity. The one with the greatest score is then selected as the best community grouping. While this is very consistent and accurate, the n! time complexity makes it impractical for anything larger than n=10.

Label Propagation(LPA)

Label Propagation Algorithm is a fast community detection method that assigns each node a unique label and iteratively updates it based on the most frequent labels among its neighbors. Over repeated updates, labels propagate through the graph, forming densely connected groups where nodes converge to the same label. LPA is computationally efficient, typically near-linear in the number of edges, but it can produce unstable or non-unique results due to an innate level of randomness within the algorithm (i.e. Random tie breaks, shuffling update order.)

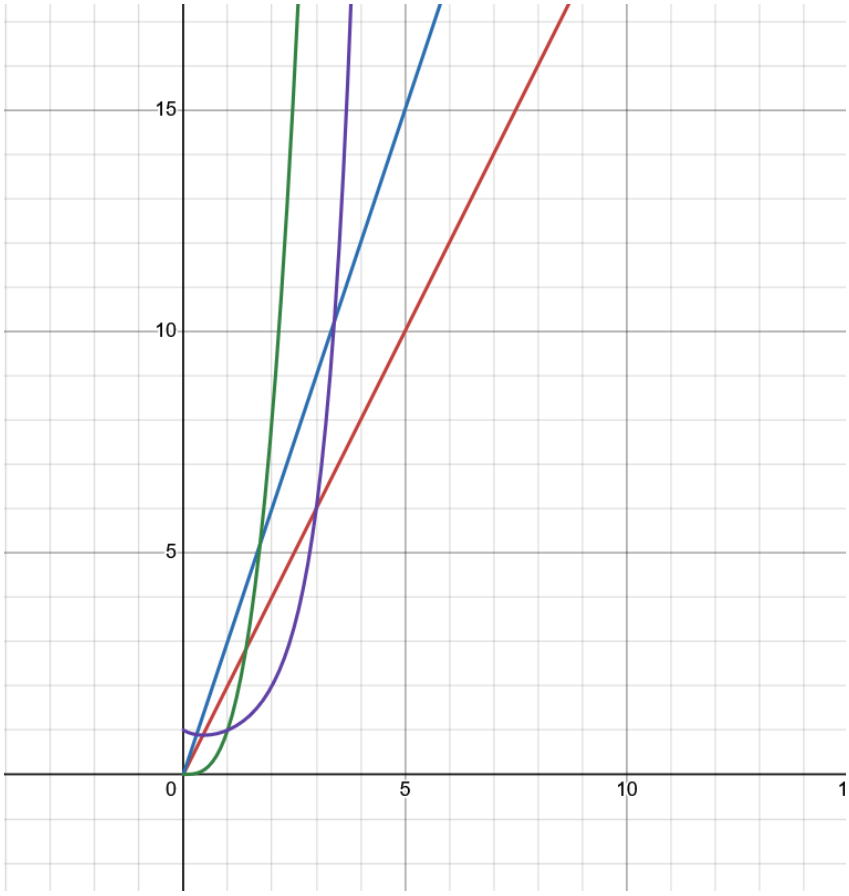
Spectral Clustering

Spectral clustering is a community detection method that uses linear algebra to identify structure in a graph. It works by constructing a matrix representation of the graph in the form of a laplacian matrix, computing its eigenvalues and eigenvectors, and then embedding nodes into a lower dimensional space. In this space, densely connected nodes end up closer together, meaning standard clustering methods (like k-means) are applied to group nodes into communities. Spectral clustering is effective at detecting non-trivial and non-convex community structures but can be computationally expensive in large dense graphs due to eigen decomposition with time complexity $O(n^3)$.

Emperor Penguin Optimization (EPO)

At the start, these assignments are generated randomly, meaning each penguin proposes a different possible way to divide the graph into communities. Over time, these candidate solutions are improved through iteration using feedback from the best performing solution, local neighborhood structure, and controlled randomness. This allows the algorithm to gradually refine a set of full graph partitions toward higher-quality community structures.

A key part of this algorithm is the temperature schedule, which controls how much the algorithm explores versus how much it focuses on improving the best solution. At the start (high temperature), the algorithm explores more by making more random changes and relying less on the current best solution. As the temperature decreases, it gradually shifts toward exploitation, where it more often copies good structures from the best solution and reinforces stable neighbor based groupings. This acts like casting a wide net early on to search many possibilities, then slowly narrowing focus to refine the best community structure over time.



***Graph showing time complexities at small input sizes. Purple is Brute Force, Red is Label Propagation, Blue is Emperor Penguin Optimization, and Green is Spectral Clustering*

Results

Below is a table that contains the results of our experimentation. All of these results are also recorded in the ./results folder in the github repository. The [number]-[number] datasets are randomly generated datasets. The first value is the number of nodes in the graph. The second number is the density percentage. A graph named “100-50” has 100 nodes with 50% graph density.

Algorithm	Dataset	Runtime(seconds)	Modularity	Communities
Brute Force	10-10%	0.003	0.22	2
Brute Force	10-50%	14.8	0.14	2
Brute Force	10-90%	16.58	0	1
Emperor Penguin Optimizer (EPO)	10-10%	0.001	0.06	5
Emperor Penguin Optimizer (EPO)	10-50%	0.004	-0.045	8
Emperor Penguin Optimizer (EPO)	10-90%	0.006	-0.093	9
Emperor Penguin Optimizer (EPO)	100-10%	0.463	0.146	28
Emperor Penguin Optimizer (EPO)	100-50%	2.732	-0.007	66
Emperor Penguin Optimizer (EPO)	100-90%	4.988	-0.00899	65
Emperor Penguin Optimizer (EPO)	Football	0.3	0.578	8
Emperor Penguin Optimizer (EPO)	Football	0.3	0.589	8
Emperor Penguin Optimizer (EPO)	Football	0.31	0.59	10
Emperor Penguin Optimizer (EPO)	Zachary's Karate Club	0.02	0.15	16
Emperor Penguin Optimizer (EPO)	Zachary's Karate Club	0.02	0.016	17
Emperor Penguin Optimizer (EPO)	Zachary's Karate Club	0.018	0.073	19
Label Propagation(LPA)	10-10%	0	0	6
Label Propagation(LPA)	10-50%	0	0	1

Label Propagation(LPA)	10-90%	0	0	1
Label Propagation(LPA)	100-10%	0.006	0	1
Label Propagation(LPA)	100-50%	0.005	0	1
Label Propagation(LPA)	100-90%	0.007	0	1
Label Propagation(LPA)	1000-10%	0.113	0	1
Label Propagation(LPA)	1000-50%	0.556	0	1
Label Propagation(LPA)	1000-90%	0.68	0	1
Label Propagation(LPA)	Football	0.002	0.558	12
Label Propagation(LPA)	Football	0.001	0.59	9
Label Propagation(LPA)	Football	0.002	0.581	13
Label Propagation(LPA)	Flower	0.006	0.965	32
Label Propagation(LPA)	Flower	0.007	0.965	32
Label Propagation(LPA)	Flower	0.007	0.965	32
Label Propagation(LPA)	Zachary's Karate Club	0.001	0.399	4
Label Propagation(LPA)	Zachary's Karate Club	0	0.36	3
Label Propagation(LPA)	Zachary's Karate Club	0.001	0.133	3
Spectral Clustering	10-10%	5.55	0	3
Spectral Clustering	10-50%	0.024	0.0274	3
Spectral Clustering	10-50%	0.008	-0.06	6
Spectral Clustering	10-90%	0.023	-0.024	3
Spectral Clustering	100-10%	0.048	0.218	3
Spectral Clustering	100-50%	0.054	0.062	3
Spectral Clustering	100-50%	0.014	0.04	6
Spectral Clustering	100-90%	0.071	0.014	3
Spectral Clustering	1000-10%	2.594	0.068	3
Spectral Clustering	1000-10%	0.423	0.047	20
Spectral Clustering	1000-50%	0.581	0.022	3
Spectral Clustering	1000-50%	0.608	0.02	6
Spectral Clustering	1000-50%	0.62	0.012	20
Spectral Clustering	1000-90%	0.83	0.0065	3
Spectral Clustering	1000-90%	0.822	0.002	20

Spectral Clustering	Flower	1.94	0.873	10
Spectral Clustering	Flower	0.317	0.963	30
Spectral Clustering	Flower	1.873	0.965	31
Spectral Clustering	Flower	0.304	0.892	50
Spectral Clustering	Football	1.589	0.601	12
Spectral Clustering	Football	0.014	0.577	6
Spectral Clustering	Zachary's Karate Club	0.008	0.273	10
Spectral Clustering	Zachary's Karate Club	0.008	-0.02	2
Spectral Clustering	Zachary's Karate Club	0.007	0.39	3

Below is a table of the average modularities for our algorithms on real life data or custom test data.

Modularity comparisons:	
Football:	**Modularity**
1. Spectral Clustering	0.589
2. EPO	0.586
3. LPA	0.576
Flower:	
1. LPA	0.965
2. Spectral Clustering	0.909
Zachary's Karate Club:	
1. LPA	0.297
2. Spectral Clustering	0.214
3. EPO	0.08

Conclusions

Based on the data from the experiments, all of the tested algorithms should work well to solve community detection problems. Their solution quality can vary based on parameters and the structure of the graph they are being applied to. Their runtimes are also as expected when compared to each other.

Empirical Analysis

The brute force algorithm was only tested on a graph of size 10 due to its n! Time complexity class. Its empirical runtime varied based on the density of the graphs it was tested on. This is likely because of the increasingly large number of permutations as the number of edges increases.

The Label Propagation algorithm had the quickest runtime, as expected. It ran at roughly the same speed as the also linear time complexity EPO algorithm. However, due to an implementation mess up in EPO, it ended up running significantly slower as graph sizes increased. More details on that in the EPO section. LPA took .68 seconds to run on a dataset of 1000 edges with 90% density, which was its longest runtime.

We expected Spectral Clustering to run slower. It has an n^3 time complexity, which should lead to much longer processing time than LPA. With that being said, in our tests, Spectral Clustering still often ran pretty fast. Its longest runtime was 5.55 seconds, which was an outlier. Most of our tests took less than a second to run, even at larger input sizes. We believe that these few experiments were outliers and the slower runtimes were caused by variables outside of the experiment (things like computer background processes) as they are definitely the results of a minority of trials. This theory is especially highlighted by the fact that the 5.55 second long run was a single run on a 10 node, 10% density graph. Every other test on a 10 node or low density graph was significantly faster. Furthermore, strangely enough, Spectral Clustering seems to run faster when its k parameter is a greater value. This isn't always true, but it is a strange pattern across our trials.

The Emperor Penguin algorithm's empirical results are skewed by our poor implementation of the algorithm. In theory, it should run roughly as fast as, if not a little bit slower than LPA. However, we messed up the implementation and ran into issues with it during experimentation on our last day of working on this project. This issue only affects runtime, not solution quality. EPO runs about roughly the same speed as LPA for graphs of size 10. However, the runtime increases significantly starting at datasets of 100 nodes. We couldn't do testing for datasets with 1000 nodes because the algorithm took too long.

Solution Quality

The randomly generated datasets were not good for determining solution qualities. Due to their random nature, they didn't really have communities, especially with densities greater than 10%. The algorithms did catch this, and gave low modularities, zero modularity, or negative modularities. These show little to no meaningful connection between nodes. This proves that the algorithms do work for detecting the lack of communities, but those experiments don't give us good data for comparing the ability TO detect communities.

To solve this problem of not being able to compare solution quality from random graphs, we retrieved three online datasets. One of them is a real dataset. The other two are specially made datasets to test graph operations. These served as much better tests to use to compare our algorithms. We used the "Zachary's Karate Club" graph, the "Football Conference (aka Football)" graph, and the "Flower" graph.

Based on the results from the modularity tables, LPA was our best algorithm. It had the highest average modularity for both the Flower graph and the Zachary's Karate Club graph. Although it scored with the lowest modularity on the Football graph, it was only slightly below the other algorithms. It still detected communities very well, only being.13 modularity points behind Spectral Clustering in modularity score. LPA did much better than expected, despite its randomness. More testing could be done in the future to determine whether LPA would be too random and inconsistent to be reliable. Averaging out the modularity scores of each run, and only doing 3 runs per dataset may have made LPA seem more consistent than it actually is.

We expected Spectral Clustering to do the best. It can do very well when given a good k value, but that is its downside. It needs to know how many communities to look for. LPA and EPO have parameters that can automatically scale, or they just don't need parameters. They can just be put onto any graph and try to detect communities without manually input parameters. Spectral Clustering however, is very dependent on being given a good k value. Looking at Spectral Clustering's results on the Zachary's Karate Club graph shows this. The graph was intended to have 2 communities, but Spectral Clustering gave a negative modularity score. It took changing that k value somewhat blindly to increase the modularity score. At that point, the average modularity score was already skewed. When the amount of communities is known, Spectral Clustering could prove quite effective. However, LPA and EPO are better when that number is unknown.

EPO couldn't be tested with the Flower graph due to its amount of nodes (almost 1000). It ran very well and consistently on the Football graph. However, its randomness did show on the Zachary's Karate Club graph when it got a randomly, much higher modularity score. EPO did incredibly poorly on that graph, essentially showing that there were no communities on that graph. It had an average modularity of .08 compared to the 214 and

.297 of the other algorithms. EPO can be good, but it has already shown to be at least somewhat inconsistent in a single run due to its randomness.

Limitations

Our main limitation for this project was time. All of our team's members are in college full time and working on top of whatever personal commitments we each have. This led to us having to rely on AI to write our EPO algorithm. Our lack of understanding of these algorithms also led to us having to leverage AI in order to work faster. We didn't fully understand EPO's implementation, which led to us having AI write that algorithm. We did, however, modify that some. Our first implementation of LPA failed due to a lack of understanding how it worked, and it had to be rewritten by a mix of human and AI programming. Both of these algorithms are now functional(although one of our changes to EPO did cause it to slow down significantly). Experimentation also took longer than expected due to the number of tests we were running. Taking the data from all of the experiments and putting that into a spreadsheet to be analyzed also took a good deal of time. Overall, time was definitely our biggest limitation. In a world where we didn't have a time constraint, we would better understand the programmatic implementation of EPO for ourselves (not just the general concepts) and debug it to make it run much faster.

Sources

Claude, ChatGPT (Used for research, explanations, debugging, and programming assistance for LPA and EPO)
https://graphia.app/example-data.html
https://networkx.org/documentation/stable/auto_examples/graph/plot_football.html
https://gist.github.com/pravj/9168fe52823c1702a07b
https://www.sciencedirect.com/science/article/pii/S1110016822005361
https://en.wikipedia.org/wiki/Label_propagation_algorithm
https://www.geeksforgeeks.org/python/community-detection-in-social-networks-using-brute-force-method/
https://networkdata.ics.uci.edu/netdata/html/zacharyKarate.html

Project Work Distribution Table

	Vincent	Wesley	Marcus
Brainstorming	35%	50%	15%
Coding	20%	75%	5%

Experiment design	90%	10%	0%
Visualization/Results	60%	30%	10%
Communication	50%	25%	25%
Submission	100%	0%	0%
Report Writing	20%	40%	40%
Presentation	10%	45%	45%

Main Difficulties

Time and Work Constraints

The main roadblock was the amount of time we had to work on this project. Our members' schedules did not line up particularly well at times, and a handful of algorithms we needed to research and learn from scratch. EPO in particular is an algorithm that was originally written to solve a different problem entirely. We adapted its core concept of exploration and exploitation to community detection, which took not a small bit of effort. AI did help us greatly in understanding these concepts and general development for our understanding.

Data Preprocessing Challenges

Data preprocessing was an initial concern for working on this project. In other words, we needed a way to take data and convert it into a graph. We found networkx, a Python library that allowed us to easily import and work with graphs. Furthermore, we discovered GML data, which could conveniently be easily imported into a graph by networkx. Using Networkx came with some tradeoffs, however. Due to the implementation of nodes within the library, we cannot customize them. In fact, nodes within the library aren't even objects. This led to having to be creative with data association for programming Label Propagation (LPA), which revolves around assigning labels to nodes. Furthermore, Python provided other libraries to help us in the development of these algorithms. While programming the Spectral Clustering algorithm, we were able to leverage the "numpy" and "sklearn" libraries to take off the overhead of programming linear algebra matrix operations and k-means clustering. Spectral Clustering is based on identifying clusters of points from a matrix representation of connectivity, so these libraries took a lot of heavy lifting off of us. We were also able to leverage AI to assist us in programming, debugging, data generation, and understanding the algorithms.

Brute Force Limitations

Brute force methods are inherently extremely inefficient. Even at a small scale of 10 nodes, this algorithm already exhibits significant slowdowns. If we were to apply it to the Zachary's Karate Club network, a dataset with only 34 nodes , it would require approximately 2.9523×10^{38} operations. Even with exascale computing performance on the order of an exaflop, the total runtime would still be on the order of 9.5 trillion years. As such this algorithm is simply here as a benchmark, and will not be included in larger test cases.